

Assignment 1 Documentation

Student: Popescu Ciprian

Group: 30431

1. Project Specification

The objective of this project is to develop an order management platform where administrators and customers can manage users, products, and orders. The application is built using Spring Boot for the backend and Angular for the frontend, following a layered architecture with RESTful API communication between the two.

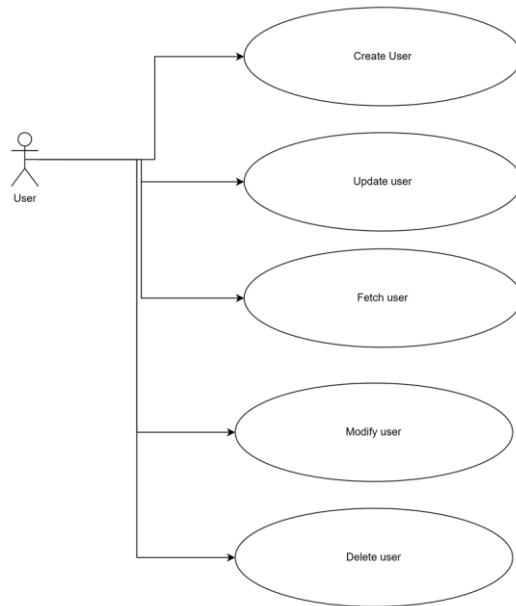
2. Requirements Analysis

Functional Requirements:

- Allow users to create, view, update, and delete Person records (with roles such as ADMIN or CUSTOMER).
- Allow the management (CRUD operations) of Product records, including tracking names and prices.
- Allow the management of Order records, mapping specific products to a specific user.

Non-Functional Requirements:

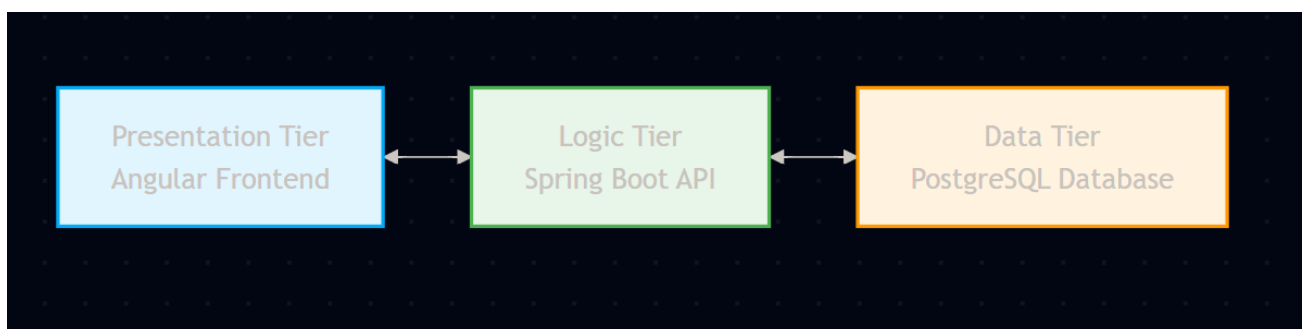
- Implement input validation (e.g., strong passwords, valid emails, and preventing duplicate emails).
- Use an ORM (Hibernate via Spring Data JPA) to handle all database interactions without writing raw SQL.
- Implement a global exception handler to intercept and return meaningful error responses for edge cases.
- Expose a RESTful API supporting full CRUD operations with appropriate HTTP methods (GET, POST, PUT, PATCH, DELETE).



3. System Architectural Design

The application follows a 3-tier architectural pattern, separating the system into three distinct layers:

- **Presentation Tier (Client):** Implemented using Angular. It is responsible for rendering the user interface and communicating with the logic tier exclusively through HTTP requests.
- **Logic Tier (Server):** Implemented using Spring Boot. It receives HTTP requests via *Controllers*, enforces business rules and validations inside *Services*, and returns responses.
- **Data Tier (Database):** Implemented using PostgreSQL and Hibernate. It is responsible for persisting data and managing the relationships between entities using *Repositories*.

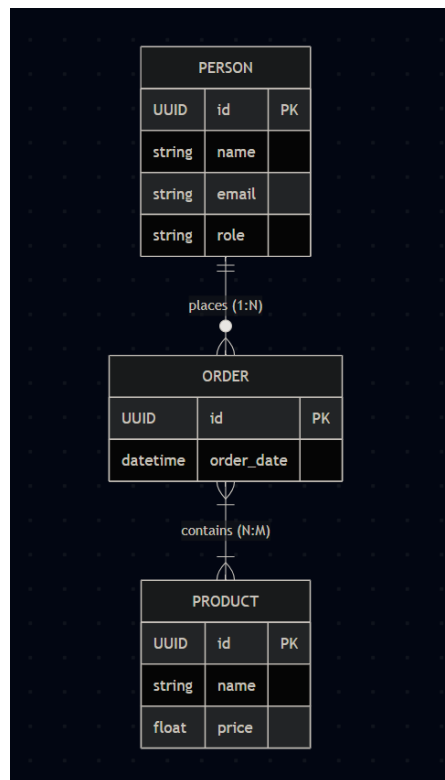


4. Data Model & Relationships

The application uses a relational database with the schema designed to support three core entities:

- **Person:** Represents a user of the platform. It stores credentials, personal details, and a Role (ADMIN/CUSTOMER).
- **Product:** Represents an item available for purchase, containing a name and price.

- **Order:** Represents a purchase event, containing a timestamp and linking a person to their chosen products.



Entity Relationships: The data tier manages the following relationships using JPA annotations:

- **One-to-Many (Person to Order):** A single person can place multiple orders. If a person is deleted, their associated orders are also removed (Cascade).
- **Many-to-Many (Order to Product):** An order can contain multiple products, and a product can be part of multiple orders. This is tracked through a join table named order_product.